
PART 1: FULLY INTEGRATED SIMULATED USERS ARCHITECTURE

Layer 1: Agent Structure

Every agent now has three distinct levels:

LEVEL 1 – STABLE IDENTITY (set at simulation start, rarely changes)

- └─ archetype # which psychological type this agent is
- └─ base_belief_position # starting position in belief space
- └─ platform_weights # how much each platform influences this agent
 - └─ imdb_weight
 - └─ reddit_weight
 - └─ youtube_weight

LEVEL 2 – STOCHASTIC PERSONALITY TRAITS (drawn fresh each timestep)

- └─ confidence_threshold ~ Normal(mean, std)
- └─ update_rate ~ Normal(mean, std)
- └─ susceptibility ~ Normal(mean, std)
- └─ open_mindedness ~ Normal(mean, std)
- └─ critical_thinking ~ Normal(mean, std)

LEVEL 3 – EMOTIONAL STATE (updates continuously during simulation)

- └─ arousal_level # 0.0 = calm, 1.0 = maximally agitated
- └─ consecutive_high_valence_count
- └─ state_decay_rate # how fast agent returns to baseline
- └─ current_cognitive_load # how mentally fatigued the agent is

The key principle is that Level 2 provides the baseline personality and Level 3 temporarily modifies it. At any given timestep, the **effective** parameters the simulation actually uses are:

```
effective_susceptibility =  
    draw(base_susceptibility) * (1 + arousal_level)
```

```
effective_critical_thinking =  
    draw(base_critical_thinking) * (1 - 0.5 * arousal_level)
```

```
effective_confidence_threshold =  
    draw(base_confidence_threshold) * (1 - 0.3 * arousal_level)
```

```
effective_update_rate =  
    draw(base_update_rate) * (1 + 0.2 * arousal_level)
```

All drawn values get clamped to valid ranges after drawing:

```
effective_value = clamp(drawn_value, min=0.0, max=1.0)
```

Layer 2: Stochasticity Integration

Stochasticity now lives in two places — agent parameters and content properties.

Agent side — already shown above. Every personality trait is a distribution, not a fixed number.

Content side — content properties also carry natural variation:

```
content.valence      ~ drawn with small noise at generation time  
content.engagement   ~ drawn with small noise at generation time
```

This means two pieces of content with the same base parameters will behave slightly differently across agents and across timesteps — which is realistic. The same video doesn't affect everyone identically.

What stochasticity produces at simulation level

Running 50 agents with identical mean parameters will now produce 50 diverging trajectories. Your results are now a distribution — you can report confidence intervals, probability of bubble formation, and variance across agent populations. This is what makes findings statistically publishable.

Layer 3: State-Dependent Emotional Variables

The emotional state machine runs as a separate update cycle after each content consumption event:

STATE UPDATE LOGIC (runs every timestep after content engagement):

```
if content.valence > 0.7: # high valence threshold  
    arousal_level += 0.15  
    consecutive_high_valence_count += 1  
    cognitive_load += 0.1  
else:  
    arousal_level -= (state_decay_rate * arousal_level)  
    consecutive_high_valence_count = 0  
    cognitive_load -= (state_decay_rate * cognitive_load)
```

```
# Hard clamp arousal to valid range
arousal_level = clamp(arousal_level, 0.0, 1.0)
cognitive_load = clamp(cognitive_load, 0.0, 1.0)
```

State thresholds and their effects

arousal_level 0.0 - 0.3: CALM state

- parameters operate near baseline
- critical thinking fully active
- confidence threshold intact

arousal_level 0.3 - 0.6: ENGAGED state

- susceptibility moderately elevated
- slight drop in critical thinking
- confidence threshold slightly lowered

arousal_level 0.6 - 0.8: AGITATED state

- susceptibility significantly elevated
- critical thinking substantially impaired
- agent accepts content further from belief position

arousal_level 0.8 - 1.0: REACTIVE state

- susceptibility near maximum
- critical thinking near zero
- agent accepts almost any high valence content
- DYNAMIC LLM TRIGGER FIRES HERE

The last state threshold — arousal crossing 0.8 — gives you your first concrete Dynamic trigger condition. This is the point where mathematical updating is insufficient and semantic interpretation of what’s happening to this agent becomes necessary.

Recovery mechanism

Agents don’t stay agitated forever. Each timestep without high valence content decays the arousal level back toward baseline:

$$\text{arousal_level}(t+1) = \text{arousal_level}(t) * (1 - \text{state_decay_rate})$$

State decay rate varies by archetype — some agents recover quickly, others ruminate and stay agitated longer. This produces the realistic pattern where some agents are permanently destabilized by a single burst of outrage content while others bounce back.

Layer 4: Multiplex Network Architecture

Agents now exist simultaneously on three network layers. Each layer operates independently but they combine to produce total social influence.

Layer A — IMDB Similarity Network

Structure: Dynamic k-nearest neighbors Connection basis: preference vector similarity
Update frequency: recalculates every N timesteps as tastes shift

```
imdb_neighbors = find_k_nearest(  
    agent.preference_vector,  
    all_agents,  
    k=10  
)  
  
imdb_social_pull = mean(  
    neighbor.belief_position  
    for neighbor in imdb_neighbors  
    if |neighbor.belief_position - agent.belief_position|  
    < agent.effective_confidence_threshold  
)
```

This layer is the only one that is truly dynamic — as an agent’s taste profile shifts through matrix factorization updates, their neighbors change, which changes who influences them socially. This creates a second-order feedback loop specific to IMDB.

Layer B — Reddit Community Network

Structure: Stochastic block model Connection basis: community membership Update frequency: fixed unless migration event occurs

```
reddit_neighbors = get_community_members(  
    agent.subscribed_communities,  
    graph_engine  
)  
  
reddit_social_pull = mean(  
    neighbor.belief_position  
    for neighbor in reddit_neighbors  
    if same_community(agent, neighbor)  
)
```

Influence within communities is strong. Influence across communities is near zero unless a cross-post event occurs. Community migration — an agent leaving one subreddit and joining another — is modeled as a discrete event triggered when an agent’s belief position drifts far enough from their community’s mean.

Layer C — YouTube Subscription Network

Structure: Sparse random graph Connection basis: subscription relationships Update frequency: fixed, rarely changes Influence weight: low — algorithm dominates

```
youtube_neighbors = get_subscriptions(agent.id, subscription_graph)  
  
youtube_social_pull = mean(  
    neighbor.belief_position
```

```

        for neighbor in youtube_neighbors
    )
    # This pull carries very low weight – see combination below

```

Combining the three layers

Total social influence at each timestep:

```

total_social_influence = (
    imdb_social_pull * agent.platform_weights['imdb'] * IMDB_LAYER_WEIGHT
+
    reddit_social_pull * agent.platform_weights['reddit'] *
    REDDIT_LAYER_WEIGHT +
    youtube_social_pull * agent.platform_weights['youtube'] *
    YOUTUBE_LAYER_WEIGHT
)

```

Default layer weights by simulation context:

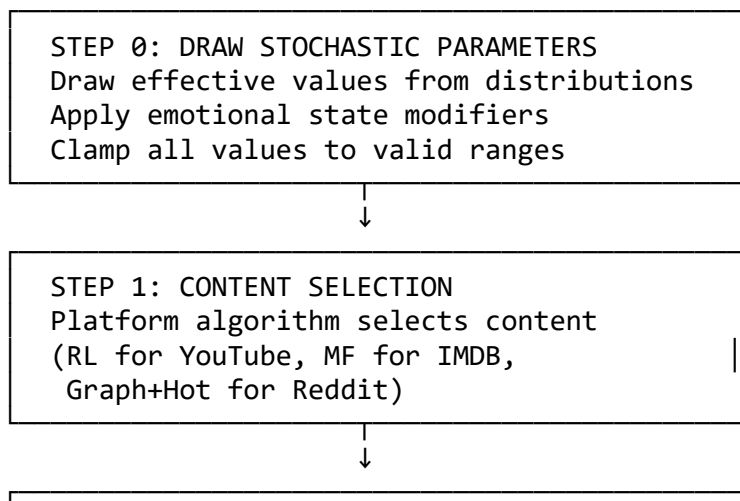
	IMDB	Reddit	YouTube
IMDB context:	0.6	0.2	0.1
Reddit context:	0.2	0.6	0.2
YouTube context:	0.1	0.2	0.7
Cross-platform:	0.33	0.33	0.33

Cross-platform mode is where your most interesting research happens — you can watch algorithmic pressure from YouTube fight social friction from Reddit in real time.

Layer 5: Complete Per-Timestep Interaction Map (Updated)

Putting everything together, one complete timestep now looks like this:

TIMESTEP N



```
STEP 2: ENGAGEMENT DECISION
distance = |content.position -
           agent.belief_position|
valence stretch:
effective_threshold += valence *
                    effective_susceptibility
if distance > effective_threshold: SKIP
```



```
STEP 3: ENGAGEMENT STRENGTH
engagement = content.engagement_potential
            * (1 - distance/effective_threshold)
            * effective_open_mindedness
            * (content.quality *
                effective_critical_thinking +
                (1 - effective_critical_thinking))
            * content.novelty
```



```
STEP 4: BELIEF UPDATE
content_influence =
    effective_update_rate
    * engagement_strength
    * (content.position -
        agent.belief_position)
```



```
STEP 5: EMOTIONAL STATE UPDATE
Update arousal based on content valence
Check if Dynamic LLM trigger fires
(arousal > 0.8 or other thresholds)
```



```
STEP 6: MULTIPLEX SOCIAL INFLUENCE
Calculate pull from all three layers
Weight by platform context
Apply combined social pull to belief
```



```
STEP 7: FINAL POSITION UPDATE
new_position = current_position
              + (content_influence * algo_weight)
              + (social_influence * social_weight)
```



STEP 8: ALGORITHM FEEDBACK

Send engagement signal back to platform
Platform updates its internal model
(RL weights / MF vectors / vote counts)



STEP 9: MEASUREMENT RECORDING

Record all six bubble metrics
Flag anomalies for Sequential Layering

TIMESTEP N+1 BEGINS

PART 2: ALGORITHM STRATEGY REVIEW — DETAILED SUMMARY

YouTube — Reinforcement Learning / Contextual Bandit

What's good — keep exactly as is

The contextual bandit approach is architecturally correct. The epsilon-greedy exploration (10% random, 90% exploitation) is realistic and necessary — without it the algorithm never discovers new content patterns. The gradient ascent weight update logic is mathematically sound. The emergent rabbit hole discovery — where the platform learns on its own that high valence content maximizes reward — is precisely the kind of emergent behavior your research needs. The learning rate of 0.05 is appropriately conservative. The weight normalization prevents explosion.

What needs to change

First, the scoring formula is incomplete. It currently uses only valence, engagement, and distance. Novelty is missing entirely, which means content never decays. Add:

```
score = (W_V * content.valence) +  
        (W_E * content.engagement_potential) -  
        (W_D * distance) +  
        (W_N * content.novelty)    # ADD THIS
```

Second, the reward signal is binary — accept or reject produces +1 or -1. This is too crude. Replace with continuous reward based on actual engagement:

```
# REPLACE:  
reward = 1 if accepted else -1
```

```
# WITH:  
reward = completion_rate * engagement_strength if accepted else -0.5
```

The softer penalty (-0.5 instead of -1) for rejection is also more realistic — the algorithm doesn't learn equally from every rejection, only from confident rejections.

Third, weight normalization clips at hard [0,1] boundaries which artificially prevents the distance penalty from becoming strongly negative. Replace hard clipping with relative normalization:

```
# REPLACE hard clipping with:  
total = sum(weights.values())  
weights = {k: v/total for k, v in weights.items()}
```

Fourth, add cold start handling for new agents entering mid-simulation:

```
if agent.interaction_count < 10:  
    # Use population average weights instead of learned weights  
    return serve_content_with_default_weights(agent, content_pool)
```

Long term research impact

The RL approach is your strongest platform mechanically because radicalization emerges without being programmed — the algorithm discovers it as an optimization strategy. Watch the learning rate carefully during implementation. Too high and agents bubble within 20 timesteps, destroying long-term trajectory analysis. Too low and nothing interesting happens. 0.05 is a reasonable start but will need empirical tuning.

IMDB/Netflix — Matrix Factorization

Correction first

You are using the IMDB dataset with Netflix's matrix factorization algorithm. The algorithm is correct. The dataset calibration needs one adjustment — normalize IMDB's 1-10 rating scale to 0-1 before feeding into the matrix factorization, so the math operates in the same range as your other parameters.

What's good — keep exactly as is

The latent factor approach is correct for taste-based recommendation. The dot product prediction formula is standard and well-validated. SGD with regularization is the right learning mechanism — the lambda regularization term prevents vector explosion which is a real risk over 1000 timesteps. The bias terms (user_bias and item_bias) are important

and correctly included — some agents rate everything generously, some movies are universally loved regardless of taste match.

What needs to change

First and most importantly, the latent vectors exist in their own mathematical space completely disconnected from your belief space. You need a bridge so that taste drift produces ideological drift:

```
# After each SGD update, add:
ideology_bleed = dot(
    agent.preference_vector,
    content.ideological_loading_vector
)
agent.belief_position += ideology_bleed * agent.effective_update_rate * 0.1
```

The 0.1 scaling factor keeps ideological bleed subtle — IMDB bubbles are soft and gradual, not dramatic. The `ideological_loading_vector` is a new content property — a mapping from the latent feature space to your belief space dimension.

Second, vector dimensionality needs specification. The document suggests 10 as an example. For IMDB this is too low — genre space alone has more than 10 meaningful dimensions. Recommend 20-30 dimensions as a balance between realism and computational speed.

Third, diversity injection has no mechanism in this model. You need to be able to force diverse content into candidate generation for intervention testing:

```
def generate_candidates(agent, content_pool, intervention=None):
    candidates = random.sample(content_pool, 100)
    if intervention == 'diversity_injection':
        diverse = get_distant_content(agent.preference_vector, content_pool,
n=20)
        candidates = candidates[:80] + diverse # Replace 20% with diverse
content
    return candidates
```

Long term research impact

Matrix factorization produces the most academically interesting bubble type — taste narrowing that is invisible to the agent. No single piece of content is extreme. No moment of radicalization occurs. The agent simply drifts into an increasingly narrow aesthetic space over hundreds of timesteps. This soft bubble is what most real recommendation research has failed to study rigorously because it requires longitudinal data. Your simulation can demonstrate it mechanically, which is a genuine contribution.

Reddit — Graph Theory + Statistical Confidence

Critical correction

Remove the C++ graph engine dependency entirely. NetworkX in Python handles everything you need — BFS traversal, community boundary enforcement, neighbor lookups — at the scale of your simulation. C++ adds implementation complexity with zero research benefit at this scale.

```
# REPLACE cpp_graph_engine calls with:
import networkx as nx
community_graph = nx.Graph()
# All graph operations through NetworkX
```

What's good — keep exactly as is

The topology-first approach is architecturally correct — Reddit's community structure is genuinely what differentiates it from other platforms. The hot sort formula is accurately implemented from Reddit's actual open-sourced algorithm. The Wilson score confidence interval is mathematically sound and correctly handles the small-sample problem. The logarithmic momentum scoring correctly captures why early upvotes are disproportionately powerful. The decay constant enforces realistic content turnover.

What needs to change

First, the scoring formula has no connection to your universal content properties. Reddit's hot score only uses votes and time — it's blind to ideological content. Add a hybrid score:

```
# REPLACE pure hot score with:
final_score = (hot_score * 0.7) +
              (ideological_alignment * community_mean * 0.3)
```

This models the real Reddit dynamic — vote momentum dominates, but community ideological alignment still influences what gets upvoted in the first place.

Second, agents are permanently locked to communities. Add a migration mechanism:

```
def check_community_migration(agent, community_graph):
    community_mean = get_community_belief_mean(
        agent.subscribed_communities[0]
    )
    drift = abs(agent.belief_position - community_mean)

    if drift > agent.effective_confidence_threshold:
        # Agent has drifted too far from community
        # Find more aligned community or leave
        migrate_or_exit(agent, community_graph)
```

Community migration is a bubble-breaking or bubble-worsening event depending on direction — an agent leaving a moderate community for an extreme one is radicalization at the social structure level. Your simulation should capture this.

Third, the Wilson score for comments needs to be integrated into the main `serve_content` pipeline rather than existing as a separate scoring path. Unified pipeline:

```
def score_content(content, current_timestep):
    if content.type == 'post':
        return hot_score(content, current_timestep)
    elif content.type == 'comment':
        return wilson_score(content)
```

Long term research impact

Reddit is your theoretically most important platform for research because it demonstrates that echo chambers form from social topology alone — no algorithmic manipulation required. When your simulation shows bubble-equivalent outcomes emerging purely from community boundary effects and vote momentum, it directly challenges the dominant narrative that platforms are actively causing polarization. That finding, demonstrated mechanically, is citable and genuinely contributes to the literature.

Summary table of algorithm changes

Platform	Keep	Fix	Add
YouTube	RL loop, epsilon-greedy, gradient ascent	Binary reward → continuous, hard clip → relative normalization	Novelty in scoring, cold start handling
IMDB/Netflix	Latent factors, SGD, bias terms, regularization	Rating scale normalization, vector dimensionality to 20-30	Ideology bleed bridge, diversity injection mechanism
Reddit	Hot sort, Wilson score, topology approach	Remove C++ dependency, permanent community lock	Hybrid ideological scoring, community migration

Do you want to move to Phase 2 — content pool design — now that Phase 1 is fully integrated and the algorithm corrections are documented?